# From Proximal Policy Optimization to Genetic Algorithm: Methods for Training Strategic Agents in RoboCup SSL

Adrian Swande*, Oskar Frej†, Gustav Samuelson‡, Lukas Bonkowski§, Ivan Blazanovic¶
School of Innovation, Design and Engineering, M.Sc.Eng Robotics
Mälardalens University, Västerås, Sweden
Email: *ase22003@student.mdu.se, †ofj22001@student.mdu.se, ‡gsn22003@student.mdu.se,
§lbi25001@student.mdu.se, ¶ibc24003@student.mdu.se

***Abstract*—The RoboCup is a set of leagues in which teams of football-playing robots compete, a project with the goal of promoting the scientific disciplines of AI and robotics. This tournament is an initative taken to advance the scientific field of AI and robotics. For this paper, two methods will be explored with the aim to create a working strategy that can score goals and defend its own in the aim to win the game. Each method will be tested using its own simulation environment. The first method utilized Proximal Policy Optimization (PPO) in a multi-agent setup with a centralized critic, similar to MAPPO, to help agents learn synchronized attacking and defending policies in simplified RoboCup SSL scenarios. The second method incorporated a genetic algorithm (GA) with a behavior tree (BT), where the BT controlled the low-level skills and the GA optimized their parameters. BT+GA method achieved more stable performance under the tested conditions compared to PPO. As future work, enhancing the stability of the simulation environments, fine-tuning low-level skills, and investigating other AI approaches are recognized as important steps in the direction of more successful strategies.**

***Index Terms*—AI, Autonomous Robots, RoboCup, Soccer**

## I. Introduction

The RoboCup is a tournament where different teams compete against each other with soccer-playing robots. The RoboCup Federation arranges several types of leagues where every league uses different types of robots in different shapes and sizes. Overall, this tournament aims to advance in the scientific field of mobile robots. This project will focus on the Small Size League (SSL), division B in particular. This specific league uses a centralised vision system that allows all robots to get information about the position of the other robots and the ball at all times. In that way, developers can focus all efforts on the strategical side of the game which makes the SSL perfect for newcomers in the RoboCup. In the SSL division B teams compete in mathces with 6 against 6 robots and the matches consist of two halves where each half is five minutes long with a five-minute pause in between. The robots are constrained to certain physical dimensions according to the rules (the robots need to fit inside a cylinder of 0.18 meters in width and 0.15 meters in height) and the robots are built by the members of each team. The playing field is 10.4 by 7.4 meters with a playing area of 9 by 6 meters and the game is played with an orange golf ball. The rules of this league are similar to regular soccer but with several modifications. For example the rules include yellow and red cards, free kicks and penalties just like in real soccer but also rules like maximum shooting speed and maximum dribbling length [1].

The aim of this project is to develop a system of robots that works well in simulation. That will be done by creating an AI system that can coordinate all six robots, handle the ball, score goals and defend against the opponents. To create this AI system two different approaches will be tested, one of them being a type of reinforcement learning and the other one being a genetic algorithm. In the long term the models we develop could be further developed and used in other works related to both RoboCup and other areas. In this paper we aim to answer the following research questions:

1. What basic skills can PPO learn in simplified RoboCup SSL scenarios?
2. How does the performance of genetically optimized skill parameters in a behaviour tree framework compare to manually selected parameters?

## II. Background

### A. Behavior Trees

Behaviour Trees (BT) are a way to structure the switching between different tasks in autonomous agents. This kind of structure was developed for controlling NPCs (non-player characters) in games and they are both modular and reactive. Modular meaning that the system consists of components that are independent and reusable, i.e. the components can be tested individually or removed without changing the whole tree. Reactive, on the other hand, means that the system adapts to changes in the surrounding space and can for example change its behaviour based on what is happening. The structure of a BT resembles a directed rooted tree with internal nodes called control flow nodes and leaf nodes called execution nodes. Each connected node are most often called parent and child where the root is the node without parents. The execution of the tree starts with the root that sends signals with a given frequency to its children to start executing and these signals are called ticks. Depending on what type a child node is, it will work differently. There are typically four

types of control flow nodes and two types of execution nodes. Starting with the control flow nodes:

- **Sequence**: routs the ticks to its own children (starting from the left) and returns failure or running to its own parent when one of its children returns failure or running. Only when it has passed all its children and every child has returned success, it returns success to its own parent.
- **Fallback**: (also called selector) works almost like a reversed version of sequence. It routs from left to right and returns success or running as long as one child returns it and returns failure only if all children return failure.
- **Parallel**: works differently from the other two, routing the ticks to all its children which means all children execute simultaneusly. The user defines how many children that needs to return succcess for the parallel node to return success to its parent. It returns failure if *N-M+1* children return failure where *M* is the user-defined amount and *N* is the number of cildren.
- **Decorator**: only has one child. It is used to check that child with special conditions, such as limited execution time, or inverting the status of that child.

For graphical representation, the sequence node is represented by a "$\rightarrow$" sign, the fallback node by a "?" sign, the parallel node by a "$\rightrightarrows$" sign, all of them with a box around the specific sign. Lastly, the decorator is represented by a rhombus. Continuing with the execution nodes:

- **Action**: executes its task and returns running while executing and success or failure when completed.
- **Condition**: checks a condition and returns either success (if its true) or failure (if its false) and never returns running.

The execution nodes are represented by a rectangle for the action node and a circle for the condition node with the specific action/condition written inside it. In this way the flow of tasks can be controlled which makes BTs a useful tool when developing an AI system [2].

### *B. Genetic Algorithm*

Genetic algorithm is a machine learning algorithm inspired by the theory of natural selection. The goal of the algorithm is to find the best possible solution to a given problem by following an evolutionary process. A population of possible solutions are tested and evaluated to measure their fitness, which is how good they are at solving the given problem. The best solutions are then brought into the next generation of solutions for further testing. The best solutions get to mate with other fitting solutions to create offspring that will have received traits from their parents. The solutions will also receive mutations to maintain genetic diversity. This entire process is then repeated until a satisfying or the best possible solution is found [3].

### *C. Reinforcement Learning*

Reinforcement learning or RL for short, is a machine learning algorithm that trains an AI model to independently make its own choices. The model learns to find the optimal policy by interacting with its environment by taking actions to find the best value function for a given policy. Through continuous training iterations it will find the best policy in regards to its value function, and in the end find the best possible solution [4]. There are three main elements and an additional optional one in reinforcement learning. These are policy, reward signal, value function and optionally a model. The policy is the current behavior of the agent. It decides on what action to take depending on the current state. The reward signal, or more generally reward, is the immediate reward that the model is given for performing a certain action. The value of the reward helps the model determine if an action was good or bad. The value function helps the model determine the best choice to take currently based on what its future rewards will look like by taking said action from its current state. The value function calculates what the best possible actions will be, to maximize its reward in the long run. The fourth and optional element is the model, but they are optional. A model-based method is based around predicting the future reward without having visited each state first, and a model-free method is based around the model learning only by itself by exploring through trial and error [5].

### *D. Proximal Policy Optimization (PPO)*

Proximal Policy Optimization (PPO) is a model-free reinforcement learning algorithm within the family of policy-gradient techniques. It enhances training stability through policy update clipping to prevent detrimental changes between iterations [6]. PPO is broadly considered a simple, yet effective baseline for continuous control and multi-agent environments. As outlined in OpenAI's Spinning Up guide, PPO aims to offer a "reliable and stable improvement step for policy gradient methods" [7]. While PPO was originally proposed as a single-agent method, it has been widely adapted for multi-agent reinforcement learning. In particular, using a centralized critic with decentralized policies (often referred to as MAPPO) has become a standard approach in domains such as robotics and multi-agent games [8]. Its efficiency, robustness, and ease of use make PPO an attractive candidate for RoboCup SSL, wherein agents are required to act in continuous spaces and coordinate in dynamic multi-agent settings.

### *E. rcssoccersim and rcssserver*

The RoboCup Soccer Simulator or rcsoccersim for short, is a tool used for research and educational purposes in regards to multiagent systems and artificial intelligence. rcssserver is a sub-repository to rcsoccersim, which contains the source code for the The RoboCup Soccer Simulator Server [9].

### *F. VMAS*

VMAS (Vectorized Multi-Agent Simulator) is a framework designed for benchmarking multi-agent reinforcement learning. It features a fully differentiable, vectorized 2D physics engine implemented in PyTorch and supports the development of custom multi-agent scenarios [10]. In this project, a new simulation model was created to replicate a football match in the B Division of the Small Size League. The virtual environment included a field and agent-based robots, all scaled to real-world dimensions. Core game mechanics—such as out-

of-bounds, goal kicks, and corner kicks—were implemented. Additionally, fundamental skills, including shooting, passing, positioning, dribbling, creating space, and throw-ins, were incorporated.

### G. Related Work

The CMDragons team won all six games they played during the RoboCup 2015 competition. In this paper they describe how they used simpler algorithms to divide their robots into defense and offense subteams to suit the state of the game. They switched between the amount of robots depending on parameters such as ball possession, field region and the aggressivness of the other team. In offense, they used algorithms to both estimate the optimal place to move for robots without the ball as well as the best action for the robot in possession of the ball. In defensive situations, algorithms were used to evaluate the threats. Both first-level and second-level threats were computed in order to stop the robot with the ball to score directly and to stop threatening passing options. Using these methods, the CMDragons were able to win the competition without conceding a single goal [11]. Due to their success, it can be useful to investigate if certain skills are more efficient to implement using simpler types of algorithms instead of using AI models.

In RoboCup, two common approaches to learning strategies are evolutionary algorithms and reinforcement learning. Early studies applied genetic algorithms (GAs) to evolve team behavior. For example, Okada and Takagi [12] demonstrated that a multi-objective GA could produce offensive, defensive, and balanced team styles in the RoboCup simulation league. Their work highlights the role of a well-designed fitness function and elitism in preserving high-quality strategies across generations, concepts also used in our GA setup.

On the reinforcement learning side, modern RoboCup teams increasingly use deep reinforcement learning. In the Small-Size League, the ZJUNlict team [13] explicitly report using Proximal Policy Optimization (PPO) to design decision-making policies, showing that PPO has become a state-of-the-art baseline in this domain. Their approach demonstrates PPO's ability to handle continuous action spaces and multi-agent dynamics, challenges also present in our SSL scenarios.

Compared to these prior works, our project applies PPO with a centralized critic in simplified SSL scenarios and combines GA with behavior trees to optimize low-level skills.

## III. Method

### A. Simulation environments

*1) rcssserver:* One of the tasks was to get a working strategy to be compatible with rcssserver so that it could be tested in a real game. The Github repository for rcssserver was cloned and set up in a virtual Linux environment by using Ubuntu. The repository for rcssmonitor was also cloned, which is the interface for watching the games in real time. rcssserver games are run by initially connecting to a server to establish a connection between the user's computer and server. Agent movement in the game is then coordinated by the server that receives commands from the connected computer on what each agent should do. The first step was to write the code for connection establishment, which used a simple UDP connection to communicate. Once that was done, the documentation for rcssserver was read to understand built in functions and commands to control the robots[14]. Firstly, a simple program was made to have the agents move into their starting positions. This program was then modified to produce different scenarios. A go-to-ball program was made where the sole purpose was for a single agent to go to the ball and shoot it towards the goal and repeat that once it scored. Each agent ran on their own thread, sending requests and commands to the servers independently for each agent.

*2) VMAS:* The VMAS framework was adapted to SSL Division B (field dimensions, ball physics, out-of-bounds and goal conditions). Each side had up to six agents. Agents could execute high-level actions (move, pass, shoot, dribble) which were translated to continuous low-level controls (e.g., dribble direction/speed, shot power, pass target scores).

### B. Reinforcement Learning

*1) Objective:* Evaluate whether PPO can learn coordinated attacking and defending behaviours in RoboCup SSL by training on simplified scenarios that reduce reward sparsity (see Background, PPO [6], [8]).

*2) Tasks / Scenarios:* To avoid sparse rewards from full matches, training used short, focused scenarios:

- **Ball at centre**: kick-off-like starts.
- **Offensive/Defensive setups**: varied distances to goal/defenders and field positions.
- **1v1 drills**: isolated attacker vs. defender.

Scenario parameters varied the number of own/opponent players and initial placements.

*3) Algorithm and Training Setup:* We applied PPO with a *centralised critic and decentralised actors* (MAPPO-style) [8]. Actors consumed per-agent observations; the critic received the joint (global) state. Representative configuration: $\approx 4$ epochs per update, batch size 32, learning rate $1 \times 10^{-3}$. Entropy/clip and other hyperparameters were kept near default values.

*4) Reward Function:* Rewards encouraged task progress and penalised failure:

- Positive: scoring, moving the ball toward the opponent's goal, approaching/controlling the ball.
- Negative: conceding a goal.

Team-level shaping for coordination (e.g., pass completion, spacing/possession) was not included in this iteration (see Discussion).

*5) Scope:* Training used short, scenario-based episodes (e.g., ball at centre, offensive/defensive, 1v1) rather than full matches to reduce reward sparsity.

### C. Genetic Algorithm

This section describes the method with which the agents for rcssserver were defined and trained.

*1) Simulation Environment:* The server *ticks* every $0.1$ seconds, wherewithin one command can be sent from each of the players and from the trainer. The program runs a loop in the background which sends a command from the trainer to the server continually (at each tick), requesting information

about the state of the field at that time step. A state $s_i \in \mathcal{S}$ where $i \in \mathbb{N}$ contains the actual field state as well as the (possibly delayed) information thereabout which is known by the actors. Field-state information include the position and velocity of the ball as well as the speed, velocity and orientation of the players.

The players are represented as a set $\mathcal{P} = \{p_1, \cdots, p_{12}\}$. A player $p \in \mathcal{P}$ is a pair $(c, f)$ where $c \in \mathcal{C} := \mathbb{N}^8$ is a vector representing the configuration of tweakable parameters for the player's actionable skills. (The parameters $c_1, \cdots, c_8 \in \mathcal{C}$ directly correspond to the in-program variables `angle_precision_far`, `angle_precision_close`, `break_distance`, `break_amount`, `angle_compensation_far`, `angle_compensation_close`, `kick_angle_margin` and `kick_distance_power_scale`.) $f : \mathcal{S} \times \mathcal{C} \to \mathcal{S}$ is the the *de facto* effect on the field state of an assigned action $\mathfrak{a} \in \mathfrak{A}$, all of which return within one tick.

At each time step $i$, a background process executes each player's "configured" task

$$F := \left\{ f_p^c \,\middle|\, f_p^c = \lambda s.f_p(s, c),\ (f_p, c) = p,\ p \in \mathcal{P} \right\}$$

which sends commands from those players to the server, resulting in the next state

$$S_i = \begin{cases} S_I & : \ i = 0 \\ \left( \bigcirc\limits_{f_p^c \in F} f_p^c \right)(S_{i-1}) & : \ \text{otherwise} \end{cases}$$

assuming compositional commutativity between players, where $S_I$ is the initial configuration of the game field.

*2) Behaviour tree:* The logic behind a player's action – i.e., what commands are chosen for sendoff to the server in a given tick – is defined by a behaviour tree generated by (1), the details of which is stated in the following paragraphs.

Let $\phi_\mathfrak{a} \in \Phi \subseteq \mathcal{S}$ be the set of all *preconditional states* of an action $\mathfrak{a} \in \mathfrak{A}$. A precondition of an action represents some statement about the set of all states $\mathcal{S}$, such as "a player cannot kick the ball unless it is near thereto", or in other words, "the set of all states where any agent $p \in \mathcal{P}$ successfully kicks the ball $\text{Kick}_p \subseteq \mathcal{S}$ is a subset to the set of all states where that player is near it $\text{NearBall}_p \subseteq \mathcal{S}$, i.e., $\text{Kick}_p \subseteq \text{NearBall}_p$".

An action $\mathfrak{a} \in \mathfrak{A}$ is either a symbol refering to some implemented function inside the program or it is a set of other actions in $\mathfrak{A}$. $\overset{\phi}{\Pi} : \mathfrak{A} \to \Phi$ is a map linking an action to its set of preconditional states. $\overset{\mathfrak{a}}{\Pi} : \Phi \to \mathfrak{A}$ is a map linking preconditional states to the action whose execution is supposed to lead thereto.

A node of the behaviour tree is represented as a pair $\langle \mathfrak{t}, \beta \rangle$ where $\mathfrak{t} \in \mathfrak{T}$ is the *type* of node – sequence, selector, condition or action – and $\beta$ is a vector (from left to right) of the nodes' children. The function $\text{BT} : \Phi \cup \mathfrak{A} \to \mathscr{B}$ (where $\mathscr{B}$ is the set of all behaviour trees) generates the appropriate behaviour tree for an action given its definition and preconditions.

$$\text{BT}(\omega) := \begin{cases} \left\langle \boxed{\to}, \left( \text{BT}\left( \overset{\phi}{\Pi}(\omega) \right), \text{BT}(\omega') \right) \right\rangle \text{ for any } \omega' \in \omega & : \ \text{Set}(\omega') \wedge \omega \in \mathfrak{A} \\ \left\langle \boxed{?}, \left( \omega, \text{BT}\left( \overset{\mathfrak{a}}{\Pi}(\omega) \right) \right) \right\rangle & : \ \omega \in \Phi \\ \langle \omega, () \rangle & : \ \text{otherwise} \end{cases} \tag{1}$$

Given by their respective definitions, the control flow nodes ensure that $\left\langle \boxed{?}, \beta \right\rangle = \left\langle \boxed{?}, (\omega_1, \omega_2) \right\rangle \Rightarrow \omega_2 \subseteq \omega_1$ and $\left\langle \boxed{\to}, \beta \right\rangle = \left\langle \boxed{\to}, (\omega_1, \cdots, \omega_N) \right\rangle \Rightarrow \overset{1}{\underset{i=N}{\subseteq}} \omega_i$. Hence, it is easy to see why the execution of a behaviour tree generated by (1) will only include states which follow the rules defined by the maps $\overset{\phi}{\Pi}$, $\overset{\mathfrak{a}}{\Pi}$ and sets $\Phi$, $\mathfrak{A}$. In this way, the behaviour tree will always choose to execute the actions which will lead to its goal. The behaviour tree generated by the action "move ball to location" is shown in Fig. 1. These precondition sets, actions and maps thereof are better illustrated by looking at the tree generated by them (Fig. 1) rather than listing them explicitly, wherefore a complete table listing them and their definitions is not provided.

*3) Training Algorithm:* During training of the agents, only their individual performance is optimized – not how they interact with each other. Perfomence is measured in the number of simulation steps it takes for an agent to reach its goal given no obsticles or opponents to which it has to adapt.

Let the monoid $M_f := (\langle \{ f \} \rangle, \circ)$ be compositionally generated by the map $f$, corresponding to some action $\mathfrak{a} \in \mathfrak{A}$. Let the set $\xi_\mathfrak{a} \subseteq \mathcal{S}$ contain all states in which the goal of the action $\mathfrak{a} \in \mathfrak{A}$ is completed. The set $V_\mathfrak{a} := \left\{ c \in \mathcal{C} \,\middle|\, \exists N \in \mathbb{N},\ \bigcirc\limits_{n=1}^{N} f^c \in \mathcal{S} \to \xi_\mathfrak{a} \right\}$ contains every configuration that can always provide a solution to $\mathfrak{a}$.

Let $\mathscr{L}_\mathfrak{a}^c$ denote the *length* of the shortest solution for a given task $\mathfrak{a}$ and configuration $c \in V_\mathfrak{a}$. A configuration $c \in \mathcal{C}$ such that $\forall \mathfrak{a} \in \mathfrak{A}$, $c \in V_\mathfrak{a}$ and $c' \in \mathcal{C} \Rightarrow \forall \mathfrak{a} \in \mathfrak{A}$, $\mathscr{L}_\mathfrak{a}^{c'} \geq \mathscr{L}_\mathfrak{a}^c$, is approximated by constraining $\mathfrak{a}$ and $\xi_\mathfrak{a}$ and optimizing $c$ therefor.

Since the action "move ball to location" is affected by all variables in the vector $c \in \mathcal{C}$, optimizing it will optimize all values in the configuration. Thus, $\mathfrak{a}$ will refer to this action hereafter, with the "completion" set $\xi_\mathfrak{a}$ defined as $L(x, y, m) = \left\{ S \in \mathcal{S} \,\middle|\, \sqrt{\left(S_{\text{ball}_x} - x\right)^2 + \left(S_{\text{ball}_y} - y\right)^2} \leq m \right\}$.

The genetic algorithm considers $\mathscr{L}_\mathfrak{a}^c$ – the *loss function* – for each $c \in G_i$, where $G_i \in \mathcal{C}^{12}$ denotes the generation $i$.

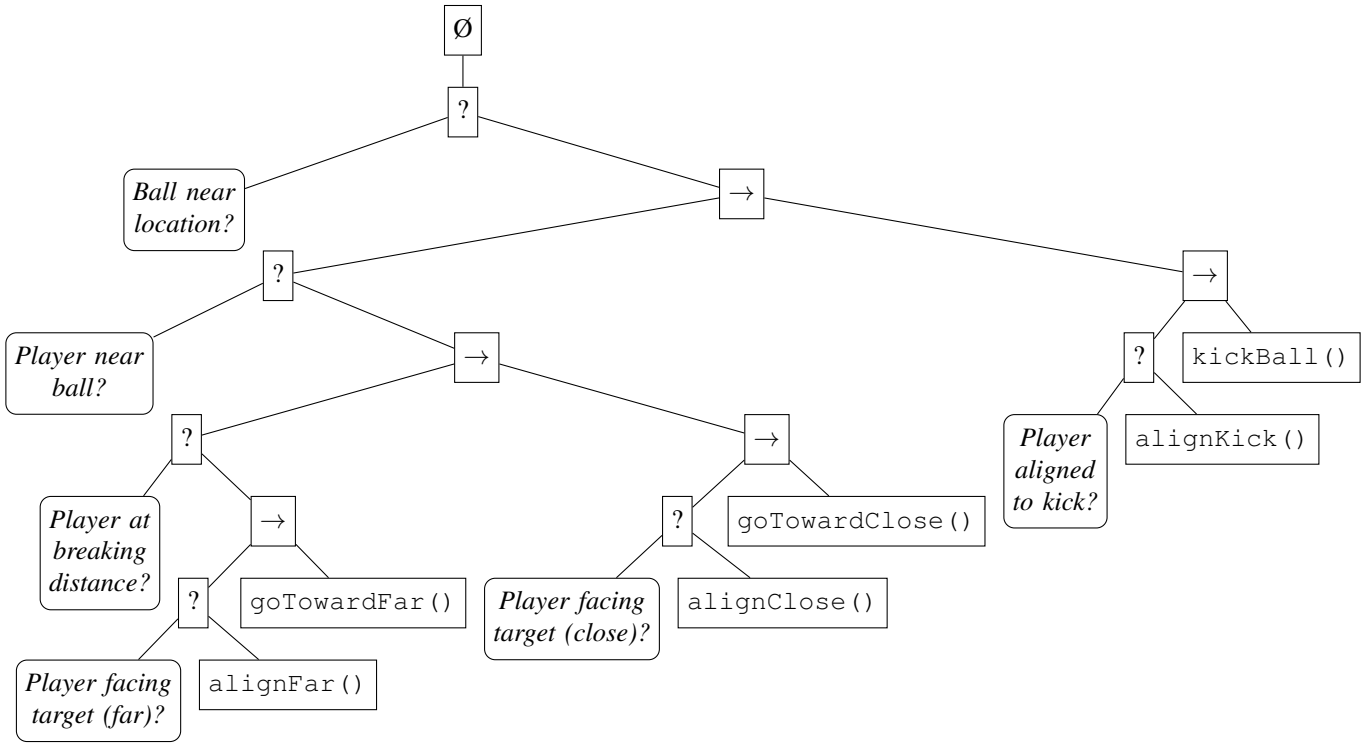



Figure 1. Behaviour tree generated by (1) for the action "move ball to location"

Notice the population size of 12 in each generation.

The first generation, $G_0$, is initialized as a collection of random configurations. The algorithm ensures via elitism that $\min_{c \in G_{i+1}} (\mathscr{L}_{\mathfrak{a}}^{c}) \leq \min_{c \in G_i} (\mathscr{L}_{\mathfrak{a}}^{c})$ given the same initial state and completion set. The $x$ and $y$ coordinates for $L$ are both uniformly randomly picked between $[-10, 10]$ for each "round" in an attempt to approximate the full set of problems $\mathcal{S} \to \mathcal{P}(\mathcal{S})$. The initial state is always $S_I$. The final fitness scores used by the algorithm to select and modify a given generation's population are the sums from 10 rounds of each individuals' loss function $L$. If an individual fails to complete the goal after 200 ticks, the euclidian distance between the ball and the desired point is added to the loss function. The algorithm uses tournament selection with 3 individuals, point crossover, and a mutation rate of 0.1.

## IV. Results

### A. Reinforcement Learning

The experiments with reinforcement learning did not give the desired results. Even in basic scenarios, the agent was not able to score goals consistently. Training in a basic two-player setup did not show any coordinated and cooperative play between the two players. The players run towards the ball slowly and dribble towards the goal. Shooting was hard to replicate even in simplified scenarios.

### B. Genetic Algorithm

In Fig. 2 and Fig. 3 are shown the fitness during the evolution of the configuration vector $c \in \mathcal{C}$ by means of the genetic algorithm described in 3. Even though the fitness seems to improve over generations and even converge, the large gap between the best fitness and the average fitness could suggest an imbalance in the training parameters regarding diversity. The best individual after around 30 generations was tested against best programmer-defined configuration in a 1 vs. 1 scenario where they were instructed to move the ball into the other player's goal. If none of the players had managed to score a goal after 60 seconds, the game was counted as a draw. In Table II is shown the programmer-defined and GA trained configurations side-by-side. The results of 25 games is shown in Table I.

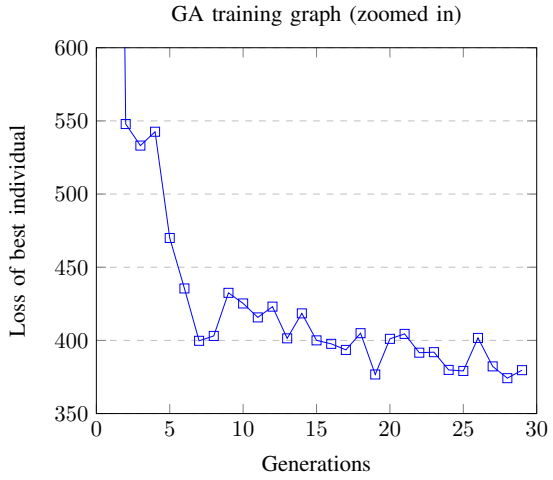



Figure 2. Graph showing the loss function of the best individual through generations.

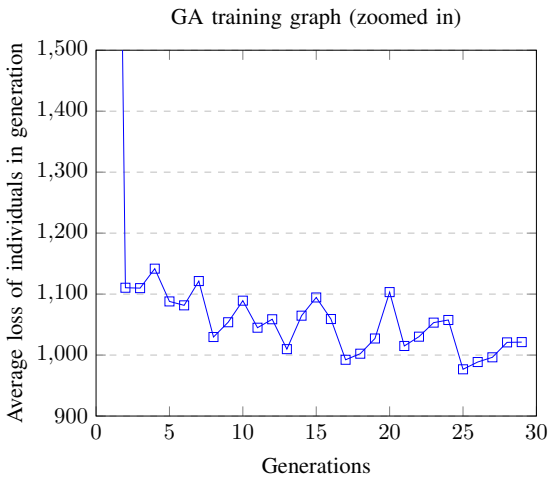



Figure 3. Graph showing the mean value of the loss function for each individual through generations.

| | | |
|---|---|---|
| GA wins | 18 | 72% |
| Programmer wins | 4 | 16% |
| Draws | 3 | 12% |

Table I
RESULTS OF GA-VS-PROGRAMMER EXPERIMENT SHOWN IN AMOUNT AND PERCENTAGE.

## V. DISCUSSION

### A. rcssserver

Although rcssserver helped producing the closest thing we had to a working strategy, the general opinion regarding rcssserver is still negative. A lot of time was spent setting up basic necessities such as the UDP connection to the server, something one would think already existed as a preset for anyone to use and not have to do themselves. Not only did it take time, once it was done we had to look through the extremely limited documentation there was. The information was very vague and left a lot of things up to the readers to figure out themselves. Complications whilst running the games also appeared, as we realized we needed proper thread management to control the flow of the game which also led to time being spent solving that problem. Due to the mentioned problems, we only managed to achieve some basic training and never got to actually apply a fully finished strategy in a real game.

| **Program variable** | **Programmer-def.** | **GA trained** |
|---|---|---|
| `angle_precision_far` | 15 | 44 |
| `angle_precision_close` | 15 | 16 |
| `break_distance` | 3 | 9 |
| `break_amount` | 50 | 33 |
| `angle_compensation_far` | 50 | 55 |
| `angle_compensation_close` | 50 | 78 |
| `kick_angle_margin` | 10 | 5 |
| `kick_distance_power_scale` | 400 | 524 |

Table II
COMPARISON OF PROGRAMMER-DEFINED AND GA TRAINED PARAMETERS.

### B. Reinforcement Learning

The results achieved by applying PPO to our VMAS RoboCup SSL setup were not as expected. Outcomes were unsatisfactory for several reasons, and further work is needed before coordinated, cooperative play can emerge.

*a) Why it did not work.*

- **Simulator mechanics:** Shooting and passing only worked in a very narrow pose/orientation window; small deviations often failed, so the same high-level choice produced inconsistent effects. This made the reward signal noisy and learning unstable.
- **Low-level skills:** High-level modes (move/dribble/shoot/pass) were mapped to hand-crafted controllers that did not always trigger predictably, so similar intents could result in weak/no kicks or unexpected ball directions.
- **Sparse shaping:** Rewards mostly covered progress/goals and lacked team-level signals (pass success, spacing, possession). Prior RoboCup work suggests rewarding successful, non-intercepted passes and penalising interceptions improves coordination [15].
- **Setup limits:** Minimal hyperparameter tuning, short training runs, and no role differentiation reduced stability and encouraged symmetric behaviour.

Given these issues, we did not include systematic learning curves or visualisations.

### C. Strategy Hierarchy

Initially, the aim was to divide the strategy into a hierarchy structure. Due to bad simulators and limited time the end product did not follow the structure that was planned from the beginning. An example of such a hierarchy is shown in Fig. 4 and examplifies how team behavior could have been organized across 5 layers, from high-level strategy down to low-level execution. Each layer builds on the one above it, enabling modular, scalable control.

*1) Game Strategy:* The top-layer (red box) defines the overall team behavior based on the given game state. Then if the team is winning and the time is lower than a specified threshold, the strategy could be set to stall. This would then provide high-level context for all other decisions made below.

*2) Play:* The play layer (yellow box) could than select coordinated maneuvers such as setting up a wing attack or forming a defensive wall. Selecting plays could be done by a decision tree based on factors like ball position, team formation, and opponent layout. Each play would then set constraints or goals for roles and tactics.

*3) Role Assignment:* This layer (green box) would dynamically assign robots to specific roles (e.g. striker, defender, goalie) based on their position, proximity to the ball, or other factors.

*4) Tactic:* The tactic layer (blue box) would define what action a robot should take in its current role. This could be whether the robot should pass, dribble, shoot, or intercept.

*5) Skill:* Lastly, the skill layer (purple box) would handle the low-level physical execution of actions. This could be moving to a position, kicking (how hard) or dribbling. Using

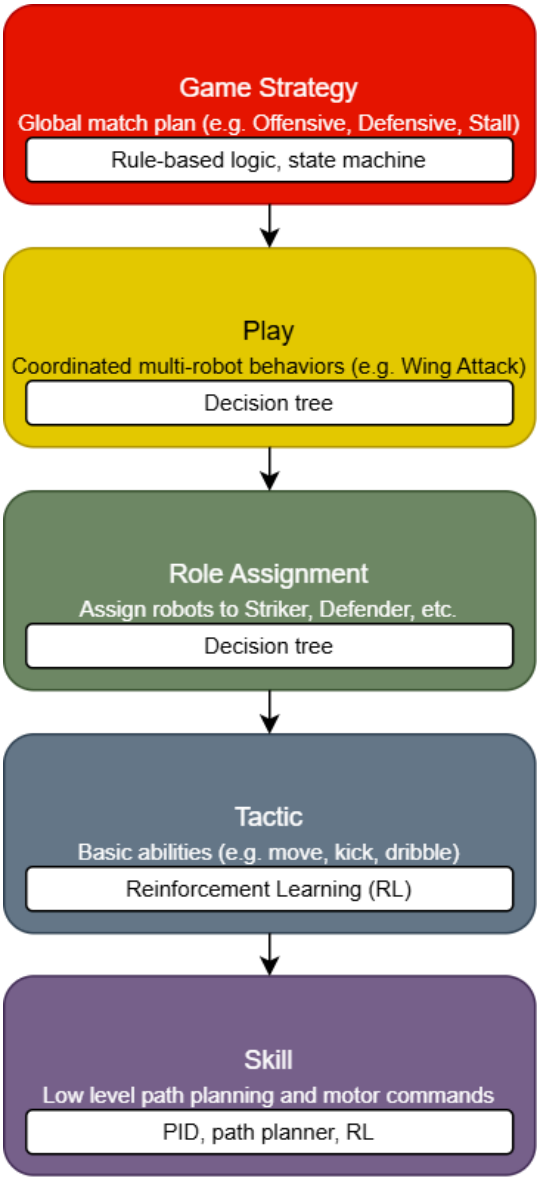



Figure 4. Hierarchical team behavior structure

a hierarchy like this one to structure the strategy is something that could be usefull for future works.

For more future work, once the simulation inconsistencies are addressed, implementing more systematic evaluation of agent performance would be suitable. This would include tracking and visualizing learning curves, episode rewards, and success rates for key behaviours such as scoring and passing. These evaluation methods are necessary for diagnosing problems and refining progress on agent results.

### D. Research Questions

To answer R.Q. 1.: PPO was able to learn the basic "go-to-ball" behavior in simplified SSL scenarios. Agents consistently approached and dribbled the ball toward the goal. However, PPO failed to learn more complex skills such as passing and shooting. The main reasons were sparse rewards, sensitive low-level mechanics, and unreliable simulation of shooting/passing.

To answer R.Q. 2.: from IV-B, it is clear that GA-optimized parameters of hard-coded skills do better than programmer-defined parameters. This was true even though the parameters of the training algorithm seem to be suboptimal – based on the training graphs. The genetic algorithm quickly (within 30 generations) found a vastly superior configuration than we did when writing the program. We speculate that the reason for this is the big configuration space which is too large for a human to search and test in a reasonable amount of time, while a computer program (the genetic algorithm) can automatically search it and test it much faster.

## VI. Conclusion

In this paper we have looked at two types of machine learning algorithms to be used for training virtual football-playing agents, with the end goal of applying the finalized and trained-on strategies on RoboCup SSL robots. For VMAS we were able to create a custom environment made for RoboCup SSL with working low-level skills. In that environment, we managed to train the agents to go to the ball using reinforcement learning but unfortunately no other training succeeded. For rcssserver, we managed to setup a working simulation environment, where we were able to create low-level skills and coordinate them via a BT. We were able to tune these skills using GA with positive results. In conclusion, GA gave the best and also most advanced results out of the two selected algorithms. PPO was able to learn basic "go-to-ball" behavior, but failed to learn more complex skills. The genetic algorithm was able to find a better configuration of variable values for hard-coded skills than the manually selected configuration found by its programmers. For future work it would be beneficial to implement more reliable shooting/passing mechanics and to improve the simulation environment as a whole. The experiment would also improve with more robust low-level skills, such as better dribbling, shooting and passing. Additionally, creating more training scenarios with simple setups like 1 against 1, 2 against 1 and pass to nearby teammate would make training more efficient. For the reinforcement learning part, creating an improved reward function that encourages real gameplay behaviour would probably improve the overall results.

## References

[1] R. O. Committee, "Robocup small size league," https://ssl.robocup.org/, accessed 16-04-2025.

[2] P. O. Michele Colledanchise, "Behavior trees in robotics and ai: An introduction," *Chapman Hall/CRC Artificial Intelligence and Robotics Series 2018*, published 2017, revised 2022.

[3] Tyler Biscontini, "Genetic algorithm (ga)," https://www.ebsco.com/research-starters/computer-science/genetic-algorithm-ga, accessed 03-06-2025.

[4] B. Jang, M. Kim, G. Harerimana, and J. W. Kim, "Q-learning algorithms: A comprehensive classification and applications," *IEEE Access*, vol. 7, pp. 133 653–133 667, 2019.

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2014. [Online]. Available: https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[7] OpenAI, "Spinning up in deep reinforcement learning: Ppo," https://spinningup.openai.com/en/latest/algorithms/ppo.html, 2018, accessed: 2025-08-16.
[8] C. Yu, A. Velu, E. Vinitsky, Y. Wang, A. Mandlekar, A. Romero, M. S. Ryoo, Y. Tian, H. Xu, and A. Anandkumar, "The surprising effectiveness of MAPPO in cooperative multi-agent games," in *Neural Information Processing Systems (NeurIPS) Deep Reinforcement Learning Workshop*, 2021. [Online]. Available: https://arxiv.org/abs/2103.01955
[9] N. Z. O. O. Herinson Rodrigues, Hidehisa Akiyama, "The robocup soccer simulator," https://github.com/rcsoccersim, accessed 05-10-2025.
[10] M. Bettini, R. Kortvelesy, J. Blumenkamp, and A. Prorok, "Vmas: A vectorized multi-agent simulator for collective robot learning," *The 16th International Symposium on Distributed Autonomous Robotic Systems*, 2022.
[11] D. Z. R. W. P. C. S. K. Juan Pablo Mendoza, Joydeep Biswas and M. Veloso, "CMDragons 2015: Coordinated Offense and Defense of the SSL Champions*," Carnegie Mellon University, Tech. Rep., 2016, accessed 21-04-2025.
[12] H. Okada and T. Takagi, "Evaluation of multi-objective genetic algorithm for robocup soccer team evolution," in *2008 IEEE Congress on Evolutionary Computation (CEC)*, 2008, pp. 1730–1737.
[13] ZJUNlict Team, "Extended team description paper," RoboCup Small Size League, Tech. Rep., 2023. [Online]. Available: https://ssl.robocup.org/wp-content/uploads/2023/02/2023_ETDP_ZJUNlict.pdf
[14] T. R. S. S. M. Committee, "The robocup soccer simulator users manual," https://rcsoccersim.readthedocs.io/en/latest/, accessed 05-07-2025.
[15] R. Wei, W. Ma, Z. Yu, W. Huang, and S. Shan, "Src 2018 team description paper," RoboCup, Tech. Rep., 2018, accessed 16-04-2025.